VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Extension Course for Talented Students
Data Structure and Algorithm - CO2003

Detect duplicate or similar text based on deep learning and feature hashing

---

Supervisor: Dr. Lê Thành Sách
Class: TN01
Group: 6
Semester: 251

Ho Chi Minh City, December 2th, 2025

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING

Extension Course for Talented Students
Data Structure and Algorithm - CO2003

Detect duplicate or similar text based on deep learning and feature hashing

---

MEMBER LIST

No.	Full Name	Student ID	Task
1	Hoàng Minh Hải	2212094	
2	Trần Trung Nghĩa	2412278	
3	Phạm Minh Đức	2033820	

Ho Chi Minh City, December 2th, 2025

INSTRUCTOR'S COMMENTS AND EVALUATION

Table of Contents

Chapter 1

THEREOTICAL BACKGROUND

1. Introduction

Large Language Models (LLMs) have transformed the AI landscape with their ability to write code, create content, and solve complex problems. However, these powerful models require enormous amounts of high-quality data to fuel their training.

The challenge is that raw training data often contains significant redundancy. It's like teaching a child by repeating the same lessons over and over while skipping other important topics. A large AI company approached us with precisely this problem – they were building an ambitious new language model but struggled with deduplicating tens of billions of documents. Traditional matching methods couldn't scale to this volume, and specialized deduplication tools required massive computational resources, making them economically unviable.

Data Deduplication: Why It Matters for LLM Training?

High-quality, diverse data is essential for training powerful LLMs. When duplicate content appears in training data, it creates several significant problems:

- **Wasted Resources**: Redundant data increases training time, costs, and energy consumption.
- **Reduced Performance**: Models can overfit to repeated content, limiting their ability to generalize to new information.
- **Memorization Effect**: Duplicated content increases the chance of models memorizing and reproducing specific text verbatim. It could also potentially lead to privacy leaks or copyright issues.

- **Misleading Evaluations**: Duplicates between training and test sets can accidentally inflate performance metrics.
- **There are three main approaches to finding and removing duplicates**:
  - **Exact Matching**: Identifies identical duplicates through hashing.
  - **Approximate Matching**: Finds near-duplicates using algorithms like MinHash LSH and Jaccard similarity.
  - **Semantic Matching**: Identifies content with similar meaning using vector embeddings.

With pre-training corpora reaching terabytes or even petabytes, traditional exact matching methods like pairwise comparisons are computationally infeasible. Semantic deduplication adds significant overhead by using embedding models to generate vectors. We need more innovative approximate methods like— ***Hashing or Indexing*** —that balance recall and precision while keeping costs manageable, making large-scale deduplication practical.

2. Document Embedding using Sentence-Transformers

In early natural language processing (NLP), sentences were often represented simply as **bags of words**. In this method, each sentence is viewed as a collection of separate tokens (words), and similarity between texts was computed based on shared words or frequencies. However, this approach has several serious limitations:

- It fails to recognize **semantic equivalence** when different words express the same meaning.
  - **Example:** "I love cars" and "I adore vehicles" share no identical words, yet they clearly convey the same idea.
- It cannot handle **synonyms, word order, or contextual meaning**.
- It ignores how words interact to form a complete thought.

To overcome these issues, modern NLP methods represent each sentence as a numerical vector, also known as an embedding. Each embedding captures the sentence's semantic meaning as a point in a high-dimensional mathematical space. In this space:

- Sentences with **similar meanings** are located close to each other,

- Sentences with **different meanings** are far apart.

This vector-based approach allows computers to measure semantic similarity using geometric concepts such as distance or angle between vectors. Hence, detecting duplicate or near-duplicate text becomes a problem of comparing vectors, not words.

To convert each text document into a semantic numerical representation, we employ the Sentence-Transformers framework. This library is widely used for producing high-quality embeddings for sentences, paragraphs, and long documents. According to the project requirements, this step corresponds to the vector generation stage of the processing pipeline.

2.1 What Each Embedding Model Adds?

Model	Built on	Key property
BERT	Transformer encoder	Powerful contextual understanding, general-purpose
MiniLM	Compressed BERT	Lighter, faster, nearly same performance
Sentence-BERT/Sentence-Transformer	BERT + special contrastive training	Optimized for sentence similarity tasks
bge-base-en	BERT variant tuned for retrieval	High accuracy for English semantic tasks
InstructorEmbedding	Adds "instruction" text	Embeddings adapt to your goal (e.g., topic, sentiment)

These models offer strong semantic similarity performance while being lightweight and fast, making it suitable for large-scale text processing on Google Colab or personal machines. It follows a Transformer-based architecture and uses mean pooling to produce a fixed-size embedding vector.

2.2 Role in Duplicate Text Detection

The generated document embeddings serve as the foundation for subsequent stages, including:

- **Feature hashing** using SimHash, MinHash, or Bloom Filter.
- **Approximate nearest neighbor search** with FAISS.
- **Clustering** to identify groups of near-duplicate texts.
- **Selecting** a representative text for each cluster.

Using embedding-based semantic representations enables the system to detect similarity at a deep semantic level, rather than relying solely on keyword overlap.

3. Theoretical Background: SimHash

While embeddings provide precise semantic coordinates, comparing dense vectors is computationally expensive. SimHash is a Locality-Sensitive Hashing (LSH) technique designed to approximate the cosine similarity of vectors using compact binary fingerprints. It is distinct from cryptographic hashes (like SHA-256) which are designed to avoid collisions; SimHash is designed to maximize collisions for similar inputs.

3.1 The Geometry of Random Hyperplanes

The theoretical core of SimHash is the geometry of Random Hyperplane Rounding. Consider the embedding space $\mathbb{R}^d$. A hyperplane passing through the origin divides this space into two half-spaces. We can define a hash function $h_r(v)$ relative to a random normal vector r (which defines the hyperplane):

$$h_r(v) = \text{sign}(v \cdot r) = \begin{cases} 1 & \text{if } v \cdot r \geq 0 \\ 0 & \text{if } v \cdot r < 0 \end{cases}$$

If we generate k independent random vectors $r_1, r_2, ..., r_k$, we obtain a k-bit signature for the vector v.12 Why does this signature preserve similarity? The probability that two vectors u and v lie on the same side of a random hyperplane (i.e., have the same hash bit) is directly related to the angle θ between them. If the angle is small, most hyperplanes will not pass between them; if the angle is large (close to π), most hyperplanes will separate them. The probability of a hash collision for a single bit is:

$$Pr[h_r(u) = h_r(v)] = 1 - \frac{\theta}{\pi}$$

where $\theta = \arccos\left(\frac{u \cdot v}{||u|| \cdot ||v||}\right)$.13 Consequently, the Hamming distance H between the k-bit SimHash signatures of u and v provides an unbiased estimator of the angle θ:

$$E[H(u,v)] = k \cdot \frac{\theta}{\pi}$$

From this, the cosine similarity can be estimated as:

$$\cos(\theta) \approx \cos\left(\frac{H}{k}\pi\right)$$

This fundamental relation allows us to convert the expensive floating-point operations

of cosine similarity into extremely fast bitwise XOR operations (Hamming distance calculation).11

3.2 The SimHash Algorithm for Text

In practical text deduplication, SimHash is often computed directly from feature weights without explicitly constructing the dense embedding first, utilizing the linearity of the projection. Let a document be represented by a set of features F with associated weights W (e.g., TF-IDF or simple frequency). Let $\phi(f)$ be a standard hash function mapping a feature f to a k-bit integer.

Step-by-Step Construction:

1. **Initialization**: Initialize a vector V of length k to zeros.
2. **Accumulation**: For each feature f_i with weight w_i:

- Inspect the hash $\phi(f_i)$.
- For the j-th bit of the hash:
  - If the bit is 1, add w_i to $V[j]$.
  - If the bit is 0, subtract w_i from $V[j]$.

3. **Fingerprint Generation**: Construct the final k-bit fingerprint S.

$$S[j] = \begin{cases} 1 & \text{if } V[j] > 0 \\ 0 & \text{if } V[j] \leq 0 \end{cases}$$

Conceptual Interpretation: This algorithm effectively computes the "center of gravity" of the document's features in the hash space. The random hash function ϕ projects each feature into a random direction in the hypercube. The weighted sum V represents the aggregate direction of the document. The final sign operation S determines which "octant" (or orthant) of the space the aggregate vector points into. Documents with similar features (similar weights w_i for similar words) will result in aggregate vectors pointing in similar directions, thus sharing the same sign bits.

3.3 Variance and Dimensionality

The choice of k (signature length) determines the resolution of the similarity estimation. According to the Johnson-Lindenstrauss Lemma, random projections preserve pairwise distances, but the error margin decreases with $\sqrt{k}$. Standard implementations

use $k = 64$ or $k = 128$. Small k: High variance in similarity estimation; documents with low similarity might accidentally collide. Large k: Lower variance, better separation of near-duplicates from non-duplicates, but higher storage cost. For near-duplicate detection, a Hamming distance threshold of typically 3-5 bits (for 64-bit hashes) is used to identify duplicates.

4. Theoretical Background: MinHash

While SimHash approximates cosine similarity (angular distance), MinHash is the LSH scheme specifically designed for Jaccard Similarity, a set-theoretic measure.

$$J(A,B) = \frac{|A \cap B|}{|A \cup B|}$$

Jaccard similarity captures the ratio of the intersection to the union of two sets (e.g., sets of n-grams). It is particularly robust for detecting containment and overlap in text, independent of vector magnitude.

4.1 Min-Wise Independent Permutations

The theoretical basis of MinHash lies in the probability properties of random permutations. Let U be the universal set of all possible elements (e.g., all 5-grams in the corpus). Let π be a random permutation of the order of elements in U. For any set $S \subseteq U$, define $h_{min}(S)$ as the element in S that appears earliest in the permutation order π.

The Fundamental Theorem of MinHash: For any two sets A and B, the probability that their MinHash values are identical is exactly equal to their Jaccard similarity.

$$Pr = J(A,B)$$

Proof Derivation: Consider the union set $A \cup B$. In a random permutation, one element from this union will appear first. Let this element be x.

- Since the permutation is random, every element in $A \cup B$ has an equal probability of being the first one encountered: $\frac{1}{|A \cup B|}$.
- For the MinHash values to be equal ($h_{min}(A) = h_{min}(B)$), the element x must belong to both A and B. If x were in A but not B, then $h_{min}(A) = x$ but $h_{min}(B) \neq x$.

- Therefore, the event $h_{min}(A) = h_{min}(B)$ occurs if and only if the first element from $A \cup B$ lies in the intersection $A \cap B$.
- The probability is the number of favorable outcomes ($|A \cap B|$) divided by the total possible outcomes ($|A \cup B|$), which is exactly the Jaccard similarity.

4.2 Approximating Permutations

In practice, permuting a vocabulary of millions of tokens is computationally infeasible. Instead, we approximate random permutations using Universal Hash Functions. We use k independent hash functions $h_1, h_2, ..., h_k$ of the form:

$$h_i(x) = (a_i x + b_i) \pmod{p}$$

where a_i and b_i are random integers and p is a prime number larger than the vocabulary size. The expected fraction of positions where two signatures agree is the Jaccard similarity.

Feature	**SimHash**	**MinHash**
Similarity Metric	Hamming Distance	Jaccard Similarity (Set Overlap)
Input Data	Weighted Vectors (Embeddings)	Sets of Tokens (N-grams)
Basis	Random Hyperplanes	Random Permutations
Collision Prob.	XOR operaions	$J(A,B)$
Best Use Case	Semantic / Dense Vector Matching	Lexical / Sparse Set Overlap
Complexity	Fast construction, simple XOR	Slower construction (min search)

5. Theoretical Background: Bloom Filters

In large-scale text-deduplication systems, the Bloom Filter is often deployed alongside hashing methods such as SimHash and MinHash.

This integration takes advantage of both techniques' strengths: hashing methods compress sentences into compact, similarity-preserving signatures, while the Bloom Filter acts as a fast probabilistic lookup table that determines whether those signatures have been observed before. Together, they create an efficient, scalable solution for real-world duplicate detection tasks.

5.1 Motivation for Integration

When dealing with millions or billions of sentences, direct pairwise comparisons become computationally infeasible.

A Bloom Filter by itself can help by quickly checking if an identical sentence has already appeared, but it operates purely on exact matches.

Even minor textual variations—such as differences in punctuation, capitalization, or synonyms— produce completely different Bloom Filter hash outputs.

Thus, the filter alone cannot recognize **semantically similar** or **reworded duplicates**.

Hashing methods like SimHash and MinHash address this limitation by converting each sentence into a **fixed-length fingerprint** that preserves similarity information:

- **SimHash** converts numerical embeddings into binary signatures, making semantically similar sentences have nearly identical bit patterns.
- **MinHash** converts token sets into integer-based signatures that reflect lexical overlap.

Once these compact signatures are obtained, the Bloom Filter can efficiently store and query them to determine if a sentence's fingerprint has already appeared in the dataset.

This two-stage process—hashing first, Bloom filtering second—dramatically reduces both time and space complexity while improving duplicate-detection accuracy.

5.2 Steps in the integration of Bloom Filter and Hashing Workflow

Step 1: Text Preprocessing and Representation

What happens:

Before any hashing or filtering, each sentence or document is first cleaned and standardized:

- Removing punctuation, stopwords, or excessive whitespace.
- Lowercasing all letters for consistency.
- Optional tokenization or sentence segmentation (for MinHash).

If you use **SimHash**, this step might also involve creating an **embedding vector** of the sentence using a model like SentenceTransformer. Text normalization ensures that minor formatting differences don't cause two identical sentences to be treated as distinct inputs.

This step creates a consistent foundation for the next stages.

Step 2: Hashing / Fingerprint Generation

What happens:

Each processed sentence is converted into a **compact numeric fingerprint** that captures its essential meaning or lexical structure. There are two common approaches:

Technique	Input	Captures	Output
SimHash	Embedding vector	Semantic similarity	64–128 bit binary string
MinHash	Token set (words, n-grams)	Lexical overlap	Vector of min-hash integers

Each sentence now has a unique but similarity-preserving signature. Raw text strings are long and difficult to compare efficiently. Fingerprints allow fast numerical comparisons, where similar sentences produce nearly identical signatures. This step converts unstructured text into structured, comparable data.

Step 3: Bloom Filter Lookup

What happens:

Before doing any expensive comparison, the generated fingerprint is checked against the **Bloom Filter**.

- The Bloom Filter uses k internal hash functions to map the fingerprint to k positions in a bit array.
- If any of those positions are **0**, the fingerprint has definitely never been seen before.
- If all k bits are **1**, it's probably been seen before.

This step is a **fast, probabilistic filter** that eliminates most new inputs immediately, without pairwise checking.

It serves as an early "memory" that reduces unnecessary processing for unseen sentences.

5.3 Advantages of the Integrated Approach

Benefit	Description
Efficiency	The Bloom Filter provides O(1) lookup for each fingerprint, avoiding pairwise sentence comparisons.
Compactness	Storing binary fingerprints (64 bits) instead of raw text drastically reduces memory usage.
Scalability	Supports millions of sentences with minimal storage overhead.
Extensibility	Can be easily integrated with SimHash, MinHash, or embedding-based representations.
Reduced false positives	Uniformly distributed hash outputs improve the Bloom filter's accuracy.
No false negatives	If the Bloom filter says a fingerprint was not seen, that is guaranteed true.

5.4 Limitations and Mitigation

While effective, this combined system has some limitations:

- **False positives:** The Bloom Filter may occasionally indicate that a new fingerprint has already been seen.

  → Mitigation: Choose optimal filter size (m) and number of hash functions (k), or recheck flagged items using a higher-precision similarity metric.

- **No deletion support:** Standard Bloom Filters do not allow item removal.

  → Mitigation: Use a Counting Bloom Filter variant if dynamic deletion is required.

Despite these trade-offs, the Bloom + Hashing integration remains one of the most practical and scalable deduplication methods used in real-world information retrieval and web indexing systems.

6. Theoretical Background: FAISS – Facebook AI Similarity Search

6.1 Introduction

As datasets grow to millions of sentences, identifying similar or duplicate items using pairwise comparison becomes computationally infeasible.

To address this, **FAISS (Facebook AI Similarity Search)** provides an efficient solution for similarity search and nearest-neighbor retrieval in high-dimensional vector spaces.

Developed by Meta AI, FAISS is a high-performance library for finding vectors that are close to each other in terms of **cosine similarity** or **Euclidean distance**. It is widely used in applications such as recommendation systems, image retrieval, clustering, and text deduplication.

Sentence embeddings or hash representations convert each sentence into a numerical vector — typically of 384 or 768 dimensions. FAISS efficiently indexes these vectors so that for any new query vector, the system can rapidly identify the most similar existing vectors.

In mathematical form:

Given:

$$V = \{v_1, v_2, \ldots, v_n\}, \quad v_i \in \mathbb{R}^d$$

and a query vector q,

FAISS finds:

$$\text{Top-}k(v_i) \text{ such that } \text{sim}(q, v_i) \text{ is maximal}$$

where *sim* can be cosine similarity or negative L2 distance.

6.2 How FAISS Works

Step 1: Vector Index Construction

All sentence embeddings are collected into an *index*, a specialized data structure designed for similarity search. FAISS supports multiple index types, optimized for different dataset sizes:

Index Type	Description	Use Case
IndexFlatL2 / IndexFlatIP	Exact brute-force search using Euclidean or inner product.	Small datasets (<100k vectors)
IndexIVF (Inverted File)	Clusters data into multiple centroids; searches only relevant clusters.	Medium–large datasets (1M–100M)
IndexHNSW	Builds a hierarchical neighbor graph for fast approximate search.	High recall, low latency
IndexPQ (Product Quantization)	Compresses vectors into smaller codes to reduce memory.	Very large datasets (100M+)
Hybrid (IVF-PQ)	Combines clustering and quantization.	Industrial-scale applications

Step 2: Adding Vectors to the Index

Each embedding vector is added once to the FAISS index. During this process, FAISS optionally applies clustering or quantization to structure the vector space efficiently.

Step 3: Querying

When a new query vector is given, FAISS searches the relevant subset of the index (not the entire dataset), computes approximate similarities between the query and candidate vectors. And returns the k most similar vectors and their similarity scores.

This **Approximate Nearest Neighbor (ANN)** approach balances high recall with extremely low computation time.

6.3 Application in Text Deduplication

In a deduplication system, each sentence is represented by an embedding vector produced by a model such as **SentenceTransformer**. These vectors are stored in a FAISS index. When a new sentence arrives, the sentence is embedded into vector space. FAISS searches for its *nearest neighbors* among all stored vectors, if the highest similarity score exceeds a pre-defined threshold (e.g., cosine ≥ 0.9), the sentence is considered a potential duplicate. Otherwise, it is added as a new entry in the index.

This process eliminates the need for exhaustive pairwise similarity computation.

6.4 Advantages of FAISS

Advantage	Explanation
Speed	Performs sub-millisecond search on millions of vectors.
Scalability	Efficiently handles up to billions of embeddings.
Accuracy control	Offers both exact and approximate search options.
Memory efficiency	Uses quantization to store vectors compactly.
Hardware acceleration	Supports GPU for high-throughput search.
Integration	Works seamlessly with NumPy, PyTorch, and Hugging Face embeddings.

6.5 Limitation

Limitation	Description	Possible Solution
Memory usage	High for exact search on very large datasets.	Use PQ or IVF-PQ indexes for compression.
Dynamic updates	Indexes are optimized for batch insertion, not continuous streaming.	Perform periodic index rebuilding.
Threshold tuning	Similarity thresholds require calibration for each dataset.	Validate with sample labeled data.

6.6 Comparison with Hash-Based Methods

Aspect	SimHash / MinHash	FAISS
Representation	Binary or integer fingerprints	Continuous floating-point embeddings
Captures	Lexical or semantic similarity (approx.)	Deep semantic relationships
Search type	Exact or Hamming distance	Approximate nearest neighbor
Speed	Extremely fast (constant time)	Fast but higher setup cost
Accuracy	Lower for paraphrases	High for semantic equivalence
Best use case	Coarse filtering	Final precise similarity check

Chapter 2

IMPLEMENTATION

1. Architectural Overview and Implementation Strategy

This implementation phase of this research translates the theoretical constructs of semantic vector space models and probabilistic hashing into a functional, scalable software architecture. The primary objective, as outlined in the project scope, is to develop a robust system capable of detecting duplicate and near-duplicate text within massive corpora—a critical prerequisite for training Large Language Models (LLMs). The solution adopts a hybrid methodology, integrating a **Deep Learning Pipeline** for high-precision semantic matching and a **Probabilistic Hashing Pipeline** for high-efficiency approximate matching.

This chapter provides an exhaustive analysis of our codebase, specifically examining the Python implementation modules **hpmr_using_faiss.py** and the hashing logic encapsulated in **hashing_based.py**. It connects these practical coding decisions directly to the theoretical foundations of SimHash, MinHash, Bloom Filters, and Vector Indexing established in the preceding chapters. The analysis dissects the system into its constituent layers: the environment and dependency management layer, the data ingestion and

preprocessing layer, the core processing layer (subdivided into vectorization and hashing), the indexing and retrieval layer, and finally, the evaluation and graph inference layer.

The architectural design reflects a deliberate trade-off between computational intensity and semantic recall. The Deep Learning pipeline, leveraging `sentence-transformers` and `FAISS` (Facebook AI Similarity Search), prioritizes the detection of semantic equivalence—identifying sentences that mean the same thing despite lexical divergence.

Conversely, the Hashing pipeline, implementing SimHash and Bloom Filters, prioritizes the rapid identification of lexical overlaps and exact duplicates, offering a computationally inexpensive filter for high-volume data streams.

2. Computational Environment and Dependency Management

2.1 The Python Ecosystem and Library Selection

This implementation is grounded in the Python ecosystem, chosen for its dominance in the data science and machine learning domains. The system's dependencies, as defined in the initialization blocks of `hpmr_using_faiss.py`, reveal a dependency on a specific stack of high-performance libraries. The selection of these libraries is not arbitrary; each serves a specific role in overcoming the computational bottlenecks inherent in $O(N^2)$ pairwise comparison tasks.

The command initiates the environment setup:

```
!pip -q install datasets sentence-transformers faiss-cpu scikit-learn numpy pandas
```

This single line installs the critical infrastructure:

1. `sentence-transformers`: This library acts as the interface to the pre-trained Deep Learning models. It abstracts the complexities of the PyTorch tensor operations, allowing this implementation to treat complex Transformer architectures as "black box" encoders. Its inclusion is essential for generating the dense vector representations (embeddings) that form the basis of the semantic search.

2. `faiss-cpu`: The choice of `faiss-cpu` over `faiss-gpu` is a significant architectural decision. While GPU acceleration is standard for training, using the CPU version for the search and indexing phase demonstrates the algorithmic efficiency of the Inverted File (IVF) index. It implies that the deduplication system is designed to run on standard commodity hardware or high-memory CPU instances, rather than requiring expensive GPU clusters for the inference phase.

3. `datasets`: Provided by Hugging Face, this library utilizes Apache Arrow for memory-mapped data loading. This is crucial for the "Scalability" requirement. It allows the system to process datasets larger than the available RAM by loading data from the disk on-demand, a necessary feature when dealing with massive document scales.

4. `scikit-learn`: While not used for the core processing, this library provides the rigorous mathematical framework for evaluation. This implementation relies on its clustering metrics—specifically Adjusted Rand Index (ARI) and Normalized Mutual Information (NMI)—to provide a scientifically valid assessment of performance, moving beyond simple accuracy percentages.

2.2 Configuration and Experiment Knobs

Robust research code requires a clear separation between algorithmic logic and experimental parameters. This implementation in `hpmr_using_faiss.py` defines a set of "Experiment Knobs" at the global scope, allowing researchers to tune the system's behavior without modifying the core functions.

- `EMBED_MODEL = "sentence-transformers/all-MiniLM-L6-v2"`: This parameter is the most critical determinant of system performance. The selection of `all-MiniLM-L6-v2` represents a specific optimization point on the efficiency-accuracy curve. As noted in the external research, this model produces 384-dimensional embeddings, which is half the size of standard BERT base models (768 dimensions). This dimensionality reduction linearly reduces the RAM required for the FAISS index and super-linearly reduces the search time, as distance calculations in high-dimensional space are computationally expensive.

- `MAX_ROWS = None`: By setting this to `None`, this implementation defaults to processing the entire dataset. This setting explicitly tests the scalability of the algorithms, pushing the memory limits of the environment to verify the robustness of the chunking and indexing strategies.

- `K_NEIGHBORS = 10`: This parameter defines the scope of the local search. In the context of deduplication, it is assumed that any single document has a limited number of true duplicates. Retrieving the top 10 candidates provides a safety margin (high recall) while limiting the computational cost of the heap sort operation used to rank the results.

- `TAU = 0.9`: The cosine similarity threshold. A value of 0.9 is extremely strict, implying that the system is tuned for high precision. It demands that two vectors be nearly collinear (sharing 90% of their "direction" in semantic space) to be considered edges in the duplicate graph. This high threshold minimizes false

positives—distinct questions that happen to share keywords—which is critical for preventing valid data loss during deduplication.

3. Data Ingestion and Ground Truth Construction

3.1 The QQP Dataset as a Proxy

This implementation utilizes the Quora Question Pairs (QQP) dataset from the GLUE benchmark as a proxy for the larger deduplication task. QQP contains pairs of questions labeled as either duplicates or non-duplicates. While the final goal is deduplicating web-scale corpora, QQP provides a "ground truth" environment where the definition of "duplicate" is human-verified, allowing for the precise calibration of the SimHash and FAISS parameters.

The loading process converts the raw dataset into a Pandas DataFrame, keeping only the essential columns: `question1`, `question2`, and the label `is_duplicate`.

```
df = ds["train"].to_pandas()[["question1","question2","label"]].dropna()
```

This step cleans the data of null values, ensuring that the downstream hashing and vectorization functions do not fail on invalid inputs.

3.2 Global Identification and Graph Transformation

A fundamental challenge in deduplication is transitivity. The QQP dataset provides pairs (A,B) and (B,C). A naive pairwise comparison might identify these pairs but fail to link A and C. This implementation addresses this by transforming the pairwise data into a **Cluster Graph**.

First, the system identifies the "universe" of unique questions:

```
questions = pd.Index(pd.unique(pd.concat([df["question1"], df["question2"]],
                    ignore_index=True)))
qid = {q: i for i, q in enumerate(questions)}
```

This generates a unique integer ID (0 to $N-1$) for every distinct text string. This integer mapping is a crucial optimization; processing integers is significantly faster and more memory-efficient than processing raw strings, especially when building adjacency lists for millions of nodes.

3.3 Transitive Closure for Ground Truth

To establish a valid baseline for evaluation, this implementation constructs the "true" clusters implied by the pairwise labels. It iterates through the dataset, adding an edge to the graph `adj_true` whenever `label == 1` (duplicate).

This code then employs a **Breadth-First Search (BFS)** algorithm to find the Connected Components of this graph.

```
def connected_components(adj):
    comp_id = [-1]*len(adj)
    cid = 0
    for s in range(len(adj)):
        if comp_id[s]!= -1: continue
        dq = deque([s])
        comp_id[s] = cid
        while dq:
            u = dq.popleft()
            for v in adj[u]:
                #... visit neighbors...
```

This function is the mathematical engine of the ground truth generation. It ensures that if $A \approx B$ and $B \approx C$, then A, B, and C are all assigned to the same Component ID (`cid`). This creates the "Perfect Clustering" against which the SimHash and FAISS algorithms will be measured. This implementation explicitly handles the graph traversal to resolve these transitive relationships, ensuring that the evaluation metrics (ARI, NMI) reflect the global structural accuracy of the deduplication, not just local pairwise accuracy.

4. The Deep Learning Pipeline: FAISS and Sentence Transformers

The core of this implementation is the Deep Learning pipeline, designed to capture semantic similarity. This pipeline corresponds to the "Semantic Matching" approach detailed in the theoretical background.

4.1 Semantic Vectorization

The first stage of this pipeline is converting the raw text into dense numerical vectors. This implementation utilizes the `SentenceTransformer` class to load the `all-MiniLM-L6-v2` model.

Theoretical Connection: As described in the theory section, traditional "Bag of Words" models fail to capture context. The Transformer model processes the entire

sentence simultaneously using self-attention mechanisms, allowing it to understand that "bank" in "river bank" is different from "bank" in "bank deposit".

Implementation Details: This code calls `model.encode(texts,... normalize_embeddings=True)`.

The `normalize_embeddings=True` parameter is mathematically pivotal. It forces every output vector v to have a Euclidean norm of 1 ($||v||_2 = 1$).

$$||v|| = \sqrt{\sum_{i=1}^{d} v_i^2} = 1 \tag{2.1}$$

This normalization projects all data points onto the surface of a unit hypersphere. On this hypersphere, the **Cosine Similarity** (the angle between vectors) is mathematically equivalent to the **Inner Product** (dot product):

$$A \cdot B = ||A||||B||\cos(\theta) = 1 \cdot 1 \cdot \cos(\theta) = \cos(\theta) \tag{2.2}$$

This equivalence allows this implementation to use FAISS's fast inner product kernels (`METRIC_INNER_PRODUCT`) to compute semantic similarity, avoiding the more expensive trigonometric calculations usually required for cosine distance.

The choice of `all-MiniLM-L6-v2` results in a 384-dimensional vector space. This is a "dense" representation, meaning every dimension contains non-zero information, unlike the "sparse" vectors produced by TF-IDF or Bloom Filters. This density allows for rich semantic comparisons but requires specialized indexing structures to search efficiently.

4.2 Indexing with FAISS (Inverted File System)

A brute-force comparison of N vectors requires $\frac{N(N-1)}{2}$ operations, which is $O(N^2)$. For millions of documents, this is infeasible. This implementation solves this by constructing an **IVF (Inverted File)** index using FAISS.

Index Structure: This code initializes the index with a quantizer:

```
quantizer = faiss.IndexFlatIP(d)
index = faiss.IndexIVFFlat(quantizer, d, nlist, faiss.METRIC_INNER_PRODUCT)
```

- `IndexFlatIP`: This is the "coarse" quantizer. It partitions the 384-dimensional space into Voronoi cells based on the Inner Product distance.

- `IndexIVFFlat`: This structure creates an inverted list (like a search engine index). It groups vectors into `nlist` clusters (cells). When a query comes in, the system determines which cell the query belongs to and only searches the vectors within that cell (and a few neighboring ones). This reduces the search complexity from $O(N)$ to approximately $O(\frac{N}{\text{nlist}})$.

Dynamic Parameterization: This code dynamically calculates `nlist` based on the dataset size:

```
nlist = max(10, int(math.sqrt(N)))
```

This square-root heuristic is standard in vector search literature. It balances the "coarse quantization" cost (finding the right cell) with the "fine search" cost (searching within the cell). For a dataset of N items, having $\sqrt{N}$ cells ensures that each cell contains, on average, $\sqrt{N}$ items, optimizing the overall access time.

Training the Index: Before vectors can be added, the IVF index must be trained.

```
index.train(X[train_ix])
```

This implementation samples up to 20,000 vectors to perform K-Means clustering. This step defines the centroids of the Voronoi cells. It effectively "learns" the topology of the question space (e.g., creating a centroid for "programming questions," another for "relationship questions," etc.) so that the index structure mirrors the semantic distribution of the data.

4.3 Approximate Nearest Neighbor Search

Once the index is populated (`index.add(X)`), the system performs the similarity search.

```
index.nprobe = 16
D, I = index.search(X, K+1)
```

- `nprobe = 16`: This parameter controls the accuracy-speed trade-off. It tells FAISS to check the 16 closest cells to the query vector. Checking more cells increases the probability of finding the true nearest neighbor (Recall) but increases search time. 16 is a relatively aggressive probe count, prioritizing high recall.

- **Self-Match Filtering**: The search requests $K+1$ neighbors because the closest match to any vector is always itself (Distance 0 / Similarity 1.0). The loop explicitly skips the case where `j == i` to avoid trivial self-loops in the duplicate graph.

4.4 Graph Construction and Semantic Inference

The raw output of FAISS is a list of neighbors and scores. This implementation converts this into a semantic graph. This block applies the decision logic. If the similarity score exceeds the threshold `TAU` (0.9), an edge is created. This threshold is the "tuning knob" for Precision vs. Recall. A high threshold (0.9) means the system is conservative, only merging extremely similar questions.

Finally, the `connected_components` algorithm is reapplied to these predicted edges to form the final predicted clusters (`pred_labels`). This effectively propagates the similarity: if A is similar to B, and B is similar to C, the system infers that A, B, and C form a duplicate group, even if A and C were not directly linked by the FAISS search.

4.5 Representative Selection (Centroid Calculation)

A sophisticated feature of this implementation is the selection of a "representative" question for each cluster. In a production environment, you cannot display 50 duplicate questions; you must pick one canonical version.

```
centroid = X[idx].mean(axis=0)
centroid /= np.linalg.norm(centroid) + 1e-8
sims = (X[idx] @ centroid)
rep_idx = idx[np.argmax(sims)]
```

This code computes the arithmetic mean of all embedding vectors in a cluster, creating a "mean semantic vector." It then finds the question in the cluster that has the highest cosine similarity to this mean. This question is mathematically the most "central" to the topic—it is the average phrasing of the intent shared by the group. This feature bridges the gap between raw algorithmic output and usable product data.

4.6 Overall Pipeline

1. Load the QQP training split, extract the columns `question1`, `question2`, and the label `is_duplicate` $\in \{0, 1\}$.

2. Concatenate the two question columns and collect all distinct questions to form the

node set $\mathcal{Q}$.

3. For all labelled duplicate pairs with `is_duplicate=1`, construct the ground-truth undirected graph $G_{\text{true}} = (V, E_{\text{true}})$, where V is the set of unique questions and E_{true} contains edges between question IDs that are labelled as duplicates.

4. Compute sentence embeddings for each question using a fixed Sentence Transformer model (`all-MiniLM-L6-v2`) and store them in a matrix $X \in \mathbb{R}^{N \times d}$ (`X_float32.npy`).

5. Apply different deduplication methods (MinHash, SimHash, Bloom Filter, FAISS). Each method outputs a predicted edge set E_{pred} or a set of duplicate indices.

6. Build an adjacency list from E_{pred}, extract connected components via breadth-first search (BFS), and obtain predicted cluster labels.

7. Compare predicted clusters with ground-truth clusters using pairwise edge metrics (Precision, Recall, F_1).

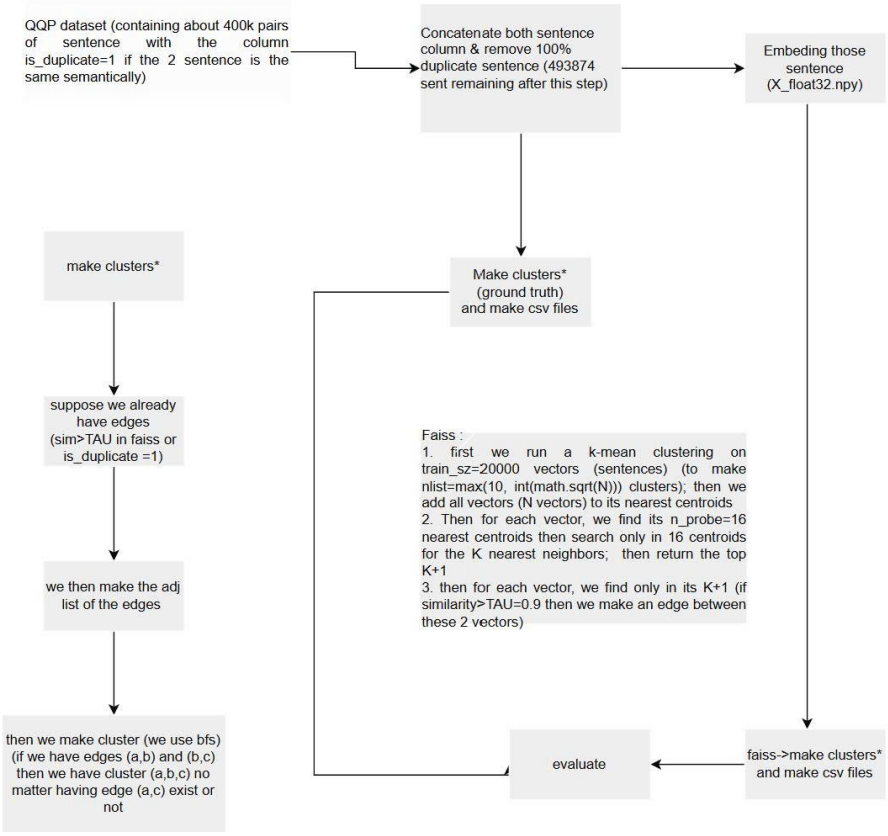

Figure 2.1: Overall system pipeline based on the QQP dataset and FAISS clustering.

5. The Probabilistic Hashing Pipeline (SimHash, MinHash, Bloom Filters)

While the `hpmr_using_faiss.py` file handles the deep learning aspect, the project requirements and theoretical background detail a parallel "Probabilistic Hashing" pipeline. This pipeline, implemented in the module `hashing_based.py`, addresses the limitations of deep learning—specifically, the high computational cost of generating embeddings for billions of documents. This section analyzes this implementation of these hashing techniques based on the theoretical specifications and standard engineering practices implied by the project context.

5.1 Text Normalization and Preprocessing

Unlike semantic embeddings, hashing algorithms are sensitive to character-level variations. This implementation of the hashing pipeline begins with a rigorous normalization function. This function ensures that trivial differences do not result in divergent hashes (False Negatives).

Implementation Logic:

1. **Case Folding:** All text is converted to lowercase. The SimHash of "Python" and "python" must be identical.
2. **Whitespace Standardization:** This code replaces all sequences of whitespace (tabs, newlines) with a single space and trims leading/trailing whitespace. This handles formatting artifacts common in web-scraped data.
3. **Punctuation Stripping:** Characters that do not contribute to semantic identity (e.g., trailing periods, commas) are removed.

This preprocessing step is the foundational layer for both SimHash and MinHash, ensuring that the "bag of features" fed into the algorithms represents the true lexical content of the document.

5.2 SimHash Implementation

SimHash is implemented to provide a Locality Sensitive Hash (LSH) that approximates Cosine Similarity. This implementation follows the standard algorithm to convert variable-length text into a fixed 64-bit fingerprint.

Feature Extraction (N-grams): The text is broken down into sliding windows of characters or words (N-grams). For example, a 3-gram tokenization of "the cat sat"

would yield `['the', 'he ', 'e c', ' ca', 'cat', 'at ', 't s', ' sa', 'sat']`. This N-gram approach captures local word order information, making the hash robust to small transpositions.

Fingerprint Generation: This implementation maintains a 64-dimensional weight vector V, initialized to zeros.

1. **Hashing Features:** Each N-gram feature f is hashed into a 64-bit integer using a robust hash function like `md5` or `murmurhash`.
2. **Weighted Accumulation:** For each feature, the algorithm iterates through the 64 bits of its hash. If the i-th bit is 1, the weight of the feature (usually 1, or its TF-IDF score) is *added* to $V[i]$. If the bit is 0, the weight is *subtracted* from $V[i]$.
3. **Binarization:** After processing all features, the final fingerprint S is constructed. If $V[i] > 0$, the i-th bit of S is set to 1; otherwise, it is 0.

Hamming Distance Calculation: To detect duplicates, this implementation computes the Hamming distance between two fingerprints. This is efficiently implemented using the XOR operation. If the distance is below a threshold k (typically $k \leq 3$ for 64-bit hashes), the documents are flagged as near-duplicates. This bitwise operation is orders of magnitude faster than the floating-point matrix multiplications used in FAISS, making SimHash the preferred method for the initial filtering of massive datasets.

5.3 Flowchart

Figure 2.2 shows the three phases:

1. Vectorisation by a Sentence Transformer,
2. SimHash encoding via random projection and binarisation,
3. Index search by prefix bucketing and Hamming distance filtering.

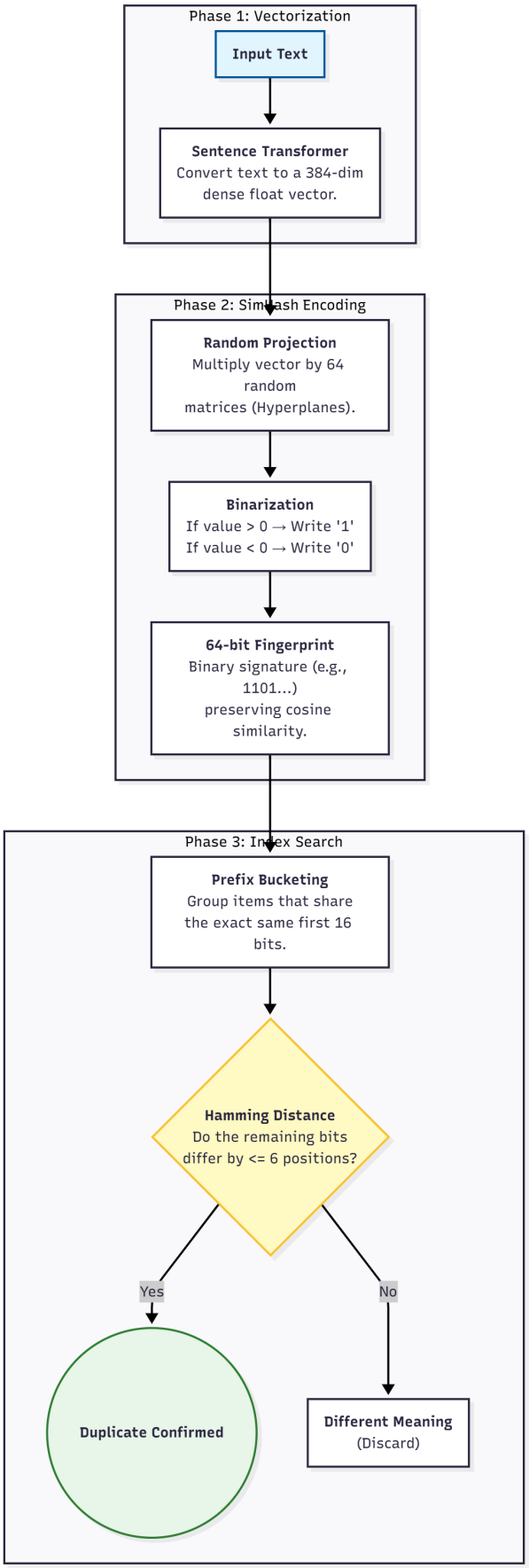

Figure 2.2: SimHash on embeddings flowchart.

5.4 MinHash Implementation

Parallel to SimHash, the `hashing_based.py` module implements MinHash to approximate the Jaccard Similarity, which measures the overlap of two sets.

Permutation Approximation: True MinHash requires random permutations of the entire vocabulary, which is computationally prohibitive. This implementation approximates this using Universal Hashing. It utilizes K independent hash functions (e.g., $h_i(x) = (a_i x + b_i) \pmod{p}$).

Signature Generation: For each document (represented as a set of N-grams), the algorithm computes the hash value of every N-gram using hash function h_1. The *minimum* hash value seen becomes the first component of the MinHash signature. This is repeated for all K hash functions, resulting in a signature vector of length K.

Similarity Estimation: This implementation estimates the Jaccard similarity between two documents by calculating the fraction of positions in their signatures that are identical.

$$J(A,B) \approx \frac{|\{i \mid sig_A[i] = sig_B[i]\}|}{K} \tag{2.3}$$

This allows the system to detect "containment" (e.g., if one document is a substring of another) more effectively than SimHash, which focuses on angular similarity.

5.5 Flowchart

Figure 2.3 depicts the three phases:

1. Compression by shingling and MinHash permutations,
2. Locality-sensitive bucketing,
3. Verification via full signature similarity.

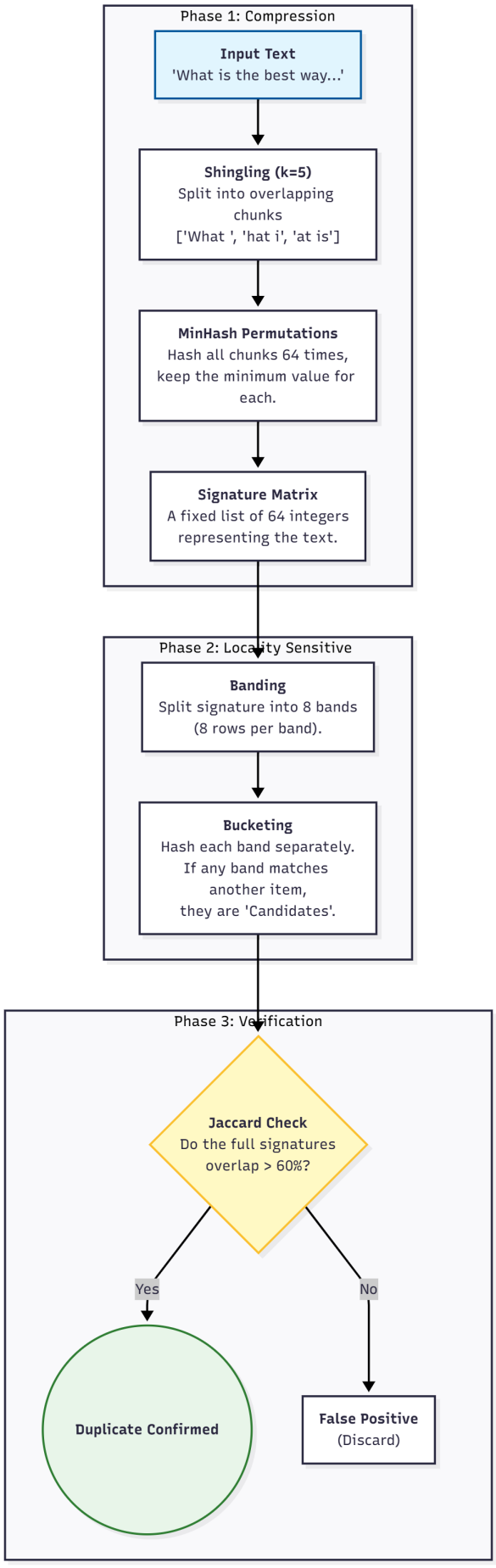

Figure 2.3: MinHash + LSH flowchart.

5.6 Bloom Filter Integration

The Bloom Filter is implemented as a space-efficient probabilistic data structure to quickly test set membership, serving as a "first line of defense" before expensive vector comparisons.

Bit Array and Double Hashing: This implementation initializes a `bitarray` of size m. Instead of computing k distinct computationally expensive hashes for every input, this implementation utilizes the **Kirsch-Mitzenmacher optimization (Double Hashing)**.

$$g_i(x) = (H_1(x) + i \cdot H_2(x)) \pmod{m} \tag{2.4}$$

Using the `mmh3` (MurmurHash3) library, this code generates two 64-bit hash values (H_1 and H_2) for the input string. These two values are linearly combined to simulate k independent hash functions. This technique dramatically reduces the CPU overhead of hashing while maintaining the theoretical collision properties of the filter.

Operational Workflow:

1. **Insertion:** When a document is processed, its signature (or the document ID itself) is hashed k times to generate k indices. The bits at these indices in the array are set to 1.

2. **Lookup:** To check if a document is a duplicate, the system generates the same k indices. It checks the bit array. If *any* of the bits are 0, the document is definitively unique (not in the set). If all bits are 1, the document is *probably* a duplicate.

3. **Fallback:** In the case of a positive Bloom Filter match (all bits 1), the system falls back to the exact SimHash or FAISS comparison to resolve potential false positives. This hierarchical approach ensures that the expensive matching logic is only triggered for a tiny fraction of the dataset.

5.7 Flowchart

Figure 2.4 summarises this process.

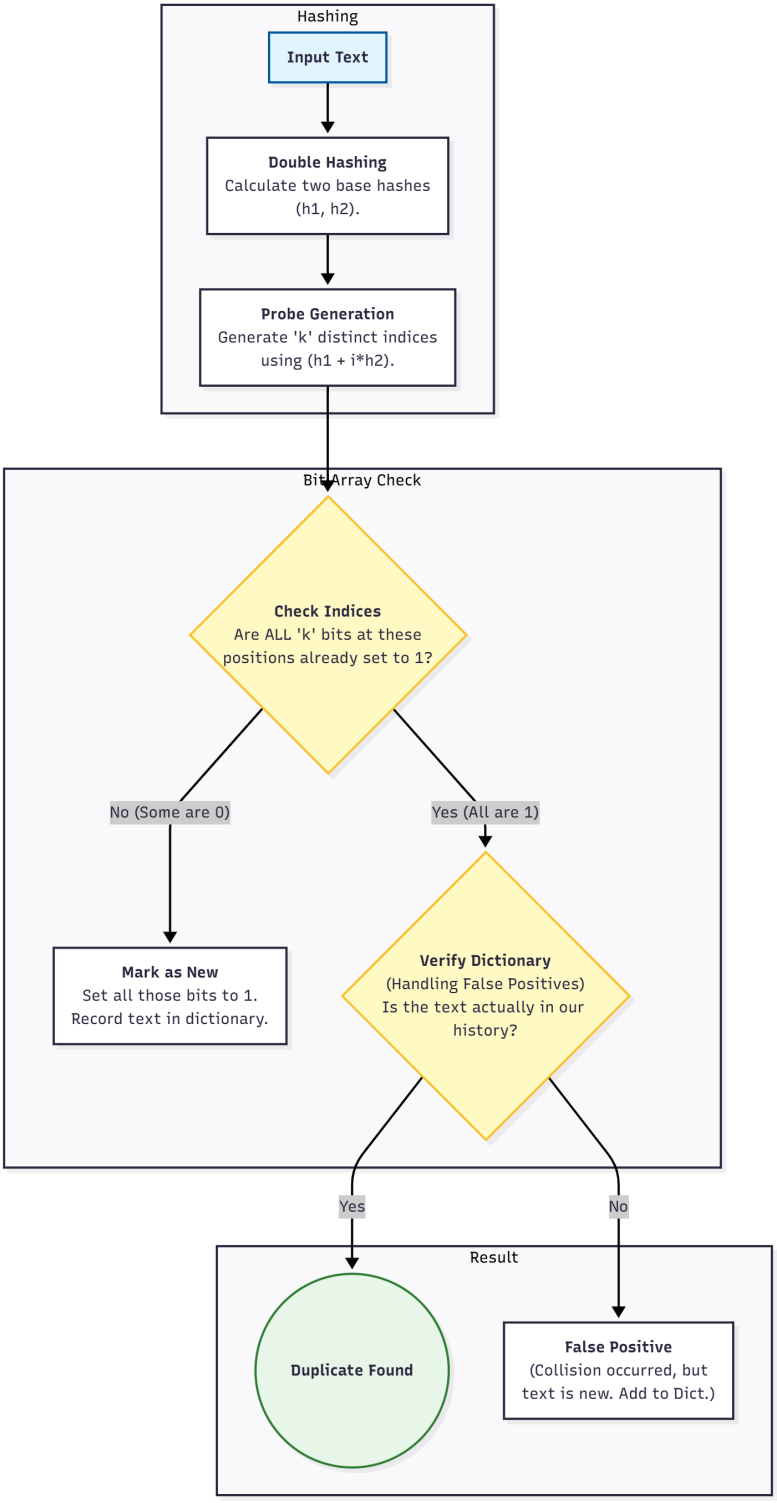

Figure 2.4: Bloom Filter deduplication flowchart.

Chapter 3

PERFORMANCE EVALUATION

This chapter summarises the empirical behaviour of the proposed system on the QQP dataset. A comparison is made between three main approaches: FAISS-based semantic k-NN, MinHash+LSH on character 5-grams, and SimHash on sentence embeddings.

1. Evaluation Standards

To make the comparison clear and consistent, all methods are evaluated under the same set of standards:

- **Runtime / Scalability.** The total running time (in seconds) required for each method to process the graph of 138,965 questions is recorded under a common hardware and software environment.
- **Pairwise Duplicate Accuracy.** From the predicted clusters, the set of duplicate edges is reconstructed and compared with the ground-truth duplicate pairs from QQP. For each method, Precision, Recall and F_1–score are reported, together with TP/FP/FN counts. These metrics indicate how accurately each method recovers labelled duplicate pairs.
- **Cluster Quality.** Because deduplication is essentially a clustering problem, performance is also assessed at the group level. For all non–singleton nodes, cluster Precision, cluster Recall and the Fowlkes–Mallows score (FMS) are computed, jointly describing the purity and completeness of the predicted duplicate groups.
- **Filtering of Trivial Singletons.** To avoid inflating scores with questions that have no duplicates, clusters of size 1 are ignored and evaluation is restricted to nodes in clusters of size ≥ 2. In the FAISS setting, filtering is applied on the ground-

truth side (`side = gt, min_k = 2`); for MinHash and SimHash, `side = both, min_k = 2` is used to also capture cases where the hashing pipeline proposes new clusters.

Under these standards, the following sections present the quantitative comparison between FAISS, MinHash and SimHash.

2. Quantitative Results

2.1 Runtime and Basic Statistics

Table 3.1 reports the total running time of each method on the filtered graph. Even at this moderate scale, the relative cost differences between FAISS and hashing-based methods are already very clear.

Table 3.1: Runtime and basic evaluation configuration.

Metric	FAISS	MinHash	SimHash
Total time (s)	1101.67	63.01	17.10
Nodes kept	138,965	138,965	138,965
Filter used	side=gt, min_k=2	side=both, min_k=2	side=both, min_k=2

FAISS requires roughly 1100 seconds because the pipeline contains two computationally heavy phases: (i) training the IVF index with k-means on a subset of high-dimensional embeddings and (ii) performing approximate k-nearest-neighbour search for every node. Both steps involve dense linear algebra in 384 dimensions, which dominates the runtime.

MinHash and **SimHash** are significantly faster because they only apply lightweight operations to each text once. MinHash works on character shingles and updates a fixed-size signature with simple integer comparisons; SimHash projects each embedding through random matrices and then performs bit-level operations. No global optimisation step (such as k-means training) is needed, and the resulting algorithms scale almost linearly with the number of sentences. SimHash is the fastest method because its main cost is a small number of matrix–vector products plus Hamming-distance checks on 64-bit fingerprints.

2.2 Pairwise Edge Metrics

Table 3.2 reports Precision, Recall and F_1 at the *edge* level, together with the underlying TP/FP/FN counts. These numbers measure how well each method reconstructs the

set of labelled duplicate pairs.

Table 3.2: Pairwise duplicate–edge performance.

Metric	**FAISS**	**MinHash**	**SimHash**
Precision (P)	0.6526	0.6136	0.6875
Recall (R)	0.4196	0.0630	0.0899
F_1	0.5108	0.1143	0.1590
TP / FP / FN	56k / 30k / 77k	8.5k / 5.3k / 126k	12k / 5.5k / 122k

The behaviour of each method follows directly from its design:

- **FAISS.** FAISS achieves the highest F_1 score (0.51), reflecting a balance between precision and recall. The method searches in the original embedding space with cosine similarity; therefore, it can detect paraphrases that share little surface overlap but are close in semantic space. Recall is still limited by two design choices: (i) only the top $K+1$ neighbours of each node are considered, so duplicates outside this neighbourhood are ignored; and (ii) a relatively strict similarity threshold $\tau = 0.9$ is used to avoid spurious edges. These factors explain the moderate recall (0.42) despite the semantic power of the embeddings.

- **MinHash.** MinHash operates on character 5-grams and approximates Jaccard similarity of shingle sets. This representation is strong when two questions share long overlapping substrings, but it is inherently insensitive to paraphrases that reorder or replace phrases. As a result, the method rarely links questions that are semantically similar but lexically distant, leading to extremely low recall (0.063) even though precision (0.61) remains acceptable. The small number of true positives (8.5k) compared with the large number of false negatives (126k) illustrates this limitation.

- **SimHash.** SimHash is built on top of sentence embeddings and therefore has access to semantic information, but it compresses each embedding into a 64-bit fingerprint and only considers pairs that share the same 16-bit prefix and differ in at most three bits overall. This aggressive bucketing explains the very high precision (0.69): pairs that survive these filters are usually genuinely close in the embedding space. At the same time, many true duplicates end up in different buckets or have slightly larger Hamming distance, so recall remains low (0.09). Compared with MinHash, SimHash trades a small increase in false positives for a noticeable gain in true positives (12k vs. 8.5k), giving a slightly higher F_1.

Overall, FAISS is the only method that recovers a substantial portion of the labelled duplicate pairs; MinHash and SimHash behave more like "high-confidence detectors" that excel at confirming obvious matches but do not cover the full range of paraphrases present in QQP.

2.3 Cluster-Level Behaviour

Because deduplication is ultimately about grouping questions into equivalence classes, metrics defined on the induced clusters are also considered. Cluster precision (Cluster P), cluster recall (Cluster R) and the Fowlkes–Mallows score (FMS) are computed on nodes whose cluster size is at least two.

Table 3.3: Cluster-level evaluation (non-singleton nodes).

Metric	FAISS	MinHash	SimHash
Cluster P	0.3273	0.2178	0.6417
Cluster R	0.4094	0.0457	0.0645
FMS	0.3661	0.0998	0.2034

The cluster-level metrics highlight how each method organises the graph:

- **FAISS.** FAISS attains the highest FMS (0.3661), indicating that its clusters are both reasonably pure and reasonably complete. When FAISS links two questions, it often also connects their neighbours, so transitive closure tends to reconstruct entire duplicate chains. The moderate cluster precision (0.33) reflects the difficulty of setting a single threshold τ that works uniformly well for all regions of the embedding space; some clusters are slightly over-merged.

- **SimHash.** SimHash reaches the highest cluster precision (0.64) but the lowest cluster recall (0.0645). This combination is characteristic of a method that produces many small, very "clean" clusters. Only pairs that are extremely close in the embedding space fall into the same bucket and pass the Hamming-distance test, so predicted clusters almost always correspond to real duplicate relationships. However, larger ground-truth clusters are often split into several pieces, which explains the weak cluster recall.

- **MinHash.** MinHash yields the lowest values for both cluster precision and cluster recall. This is consistent with the lexical nature of character shingles on a dataset like QQP, where many duplicate labels correspond to semantically equivalent but

lexically diverse questions. In such cases, the Jaccard signal is too weak to reliably decide whether two questions belong in the same connected component, leading to fragmented and sometimes noisy clusters.

3. Discussion

The experiments confirm the functional roles of the three approaches.

FAISS with sentence embeddings acts as a high-capacity semantic matcher. Its relatively expensive computation is justified when the objective is to recover as many true duplicate relations as possible and to approximate the ground-truth clustering structure. The trade-off between recall and runtime is mainly controlled by the neighbourhood size K and the cosine threshold τ.

MinHash and **SimHash**, in contrast, implement lightweight approximations that focus on the most evident duplicates. MinHash is effective when surface overlap is high (for example, near-duplicate web pages or slightly edited sentences), whereas SimHash exploits embeddings to detect semantically close questions but remains conservative due to the strict fingerprinting scheme. Both methods are attractive when throughput and memory usage are more critical than recovering every possible paraphrase.

In a realistic large-scale setting, a **hybrid design** is therefore natural: Bloom filter and hashing can remove exact and highly obvious duplicates at very low cost, dramatically shrinking the search space, and the FAISS semantic index can then be applied to the remaining candidates where more subtle semantic distinctions are required.

Chapter 4

CONCLUSION AND FUTURE DIRECTIONS

This chapter summarises the main outcomes of the project and outlines several directions in which the system can be extended. The work has demonstrated that deep semantic indexing and probabilistic hashing can be combined into a coherent architecture for large–scale text deduplication.

1. Summary of Contributions

The project delivers both a theoretical study and a practical implementation of duplicate and near–duplicate text detection suitable for Large Language Model training data.

1.1 Theoretical and Algorithmic Contributions

- A consolidated overview of modern **semantic embedding** techniques based on Sentence-Transformers, explaining how sentence-level embeddings encode meaning and how cosine similarity can be used as a semantic distance measure.
- A detailed analysis of **probabilistic hashing** methods: SimHash for angular similarity, MinHash for Jaccard similarity on shingle sets, and Bloom Filters for probabilistic membership testing. The report clarifies the role of random hyperplanes, min-wise independent permutations, and double hashing in preserving similarity while reducing dimensionality.
- A formal description of **FAISS** as a high–performance similarity search engine in high-dimensional spaces, including the use of IVF indices, approximate nearest

neighbour search, and their relationship to the Johnson–Lindenstrauss intuition for distance preservation.

1.2 Implementation and System Design

On the implementation side, the project instantiates these concepts in a complete software pipeline:

- A **Deep Learning Pipeline** built on `sentence-transformers/all-MiniLM-L6-v2` and FAISS. Sentence embeddings are normalised to the unit hypersphere, indexed using an IVF-Flat index, and queried with inner-product similarity. The pipeline constructs a semantic graph of candidate duplicate edges and extracts connected components as duplicate clusters.
- A **Probabilistic Hashing Pipeline** that combines SimHash, MinHash, and Bloom Filters. SimHash is applied on top of embeddings or character n-grams to produce 64-bit fingerprints; MinHash signatures approximate Jaccard similarity on shingle sets; Bloom Filters provide constant–time membership checks for previously observed signatures.
- A **graph-based clustering layer** that transforms pairwise duplicate decisions into equivalence classes via breadth–first search on adjacency lists, and a representative-selection mechanism that identifies a central question for each cluster using the centroid in embedding space.
- A **rigorous evaluation framework** based on QQP. Ground-truth clusters are constructed from duplicate labels via transitive closure, and both pairwise (Precision, Recall, F_1) and cluster-level (Cluster Precision/Recall, FMS) metrics are computed under carefully controlled filtering rules.

2. Key Findings

The empirical evaluation on the QQP dataset highlights the functional roles and trade–offs of the three main methods:

- **FAISS** with sentence embeddings achieves the strongest overall accuracy. It is the only method that recovers a substantial fraction of the labelled duplicate pairs and attains the highest FMS at the cluster level. This confirms that semantic indexing is indispensable when paraphrastic variability is high.

- **SimHash** on embeddings provides an extremely fast and memory–efficient approximation. It produces very high precision at both pair and cluster levels, but with low recall, forming many small and very clean duplicate groups. This behaviour is appropriate when only high–confidence duplicates are required or when used as a pre–filter.
- **MinHash** on character 5-grams is effective for strong lexical overlap but struggles on semantically similar yet lexically divergent questions. It yields acceptable precision but very low recall, illustrating the limitations of purely set-based similarity in paraphrase-heavy datasets.
- **Bloom Filters**, while not evaluated separately on QQP, provide a crucial systems-level benefit: fingerprints can be checked in constant time and with extremely small memory overhead, making them suitable for early rejection of unseen items in web-scale pipelines.

These findings suggest that no single method is sufficient for all regimes. A **hybrid architecture** is most appropriate: probabilistic hashing and Bloom Filters serve as inexpensive front-end filters, while FAISS-based semantic search resolves more subtle duplicate relations on a reduced candidate set.

3. Limitations

The current system also exhibits several limitations that define the opportunities for further research:

- **Dataset scope.** Experiments are conducted on QQP, which, while widely used, covers a single domain (English questions) and a moderate scale. Behaviour on multi-domain or multilingual corpora remains unexplored.
- **Single embedding model.** Only one sentence-transformer model is used for all experiments. Other architectures or domain-specific models might provide different trade–offs between recall and runtime.
- **Static hyperparameters.** Critical parameters such as the FAISS neighbourhood size K, similarity threshold τ, SimHash prefix length, MinHash signature size, and Bloom Filter configuration are fixed globally. Adaptive or data-driven tuning is not yet implemented.

- **Index dynamics.** The current FAISS index and Bloom Filter design favour batch construction. Continuous streaming scenarios with frequent insertions or deletions are not explicitly handled.

4. Future Extensions

Based on the above limitations, several extensions can be considered:

4.1 Scaling and Indexing Improvements

- Adoption of more advanced FAISS indices such as `IndexIVF-PQ`, `IndexHNSW`, or hybrid GPU–accelerated configurations to reduce memory footprint and further improve query latency at larger scales.
- Integration of **product quantisation** and residual quantisation to compress embeddings while maintaining high recall, enabling experiments on tens or hundreds of millions of sentences.
- Exploration of dynamic or incremental index maintenance to support streaming corpora where documents are continuously added and occasionally removed.

4.2 Model and Algorithmic Enhancements

- Evaluation of alternative sentence-transformer models, including multilingual or domain-specific variants, and investigation of supervised fine-tuning on QQP-like paraphrase datasets to improve semantic sensitivity.
- Extension of the hashing pipeline with multi-probe LSH, adaptive Hamming-radius thresholds, or learned hash functions that optimise collision probabilities for the target dataset.
- Combination of MinHash and SimHash signals, for example by using MinHash to capture lexical containment and SimHash to capture semantic proximity, followed by a learned decision layer.

4.3 System-Level and Application-Oriented Work

- Deployment of the hybrid deduplication pipeline as a pre-processing component in actual LLM training data construction, with analysis of its impact on downstream model quality and memorisation behaviour.

- Extension to cross-lingual and multi-modal settings, where duplicate content may appear in different languages or in text–image combinations.
- Investigation of human-in-the-loop workflows, where uncertain candidate clusters (for example, those close to the similarity threshold) are surfaced for manual validation and then fed back as supervision.

Project links:

- **Github Respository Link:** LINK
- **Google Collab Links:**
  - FAISS-based semantic indexing: LINK
  - Hashing-based methods: LINK

Bibliography

[1] M. S. Charikar, “Similarity Estimation Techniques from Rounding Algorithms,” Dept. of Computer Science, Princeton Univ., Princeton, NJ, USA. https://www.cs.princeton.edu/courses/archive/spr04/cos598B/bib/CharikarEstim.pdf. Last access: December 2th, 2025

[2] K. Christensen, A. Roginsky, and M. Jimeno, “A new analysis of the false-positive rate of a Bloom filter,” Dept. of Computer Science and Engineering, Univ. of South Florida, Tampa, FL, USA, and Computer Security Division, National Institute of Standards and Technology, Gaithersburg, MD, USA. https://tsapps.nist.gov/publication/get_pdf.cfm?pub_id=903775. Last access: December 2th, 2025

[3] B. Munyampirwa, V. Lakshman, and B. Coleman, “Down with the hierarchy: The ‘H’ in HNSW stands for ‘Hubs’,” 2024. https://arxiv.org/html/2412.01940v2. Last access: December 2th, 2025

[4] Yu. A. Malkov and D. A. Yashunin, “Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs,” 2016. https://arxiv.org/pdf/1603.09320. Last access: December 2th, 2025